



Catálogo de Metadados

Como a Plataforma de Dados MinC registra, para cada tabela produzida, onde ela vive, como foi materializada, o que significa e quando foi atualizada pela última vez — com a cobertura conferida diretamente no banco de dados.

Universidade de Brasília

Lab Livre · Faculdade do Gama, UnB

Ministério da Cultura

Gov Hub BR

Projeto de Pesquisa

Plataforma de Dados do Ministério da Cultura · Gov Hub BR

Meta 02 · Produto 3

Catálogo de metadados da plataforma.

Documento

Catálogo de Metadados



Índice

01	Objetivo e escopo	4
	O que é um catálogo de metadados aqui	4
	Escopo	4
02	Como o catálogo é gerado	5
	A varredura do grafo	5
	Carga incremental com chave composta	5
	Quando o catálogo é atualizado	6
	Integração com catálogo externo	6
03	Campos do catálogo	7
04	Cobertura conferida no banco	8
	Distribuição por domínio e camada	9
	Modelos que aguardam a fonte para entrar no catálogo	9
05	Limites e evolução	10





Este documento descreve o catálogo de metadados da Plataforma de Dados MinC: o que ele registra, como é produzido, o que já cobre hoje e o que ainda falta para atender integralmente ao produto previsto na Meta 02.

Nesta versão, os números de cobertura foram conferidos **diretamente no banco de dados**, e não apenas nos arquivos do repositório.

O que é um catálogo de metadados aqui

Metadado é o dado sobre o dado: não o valor de um pagamento, mas em que schema aquela tabela mora, como foi construída, o que ela significa e em que momento foi calculada pela última vez. Sem isso, quem consulta o banco encontra tabelas sem saber se estão frescas, de onde vieram ou se podem ser citadas.

Na Plataforma de Dados MinC esse registro não é uma planilha mantida à mão. É uma tabela do próprio banco, `metadata.models_metadata`, reescrita a cada execução do dbt a partir do grafo interno do projeto. O catálogo, portanto, não pode divergir do que existe: ele é derivado do que existe.

POR QUE ISSO IMPORTA

Um catálogo escrito à mão envelhece em silêncio: alguém renomeia um modelo, ninguém atualiza a planilha, e a documentação passa a descrever um banco que não existe mais. Um catálogo derivado do grafo do dbt não tem como envelhecer: se o modelo sai do projeto, ele sai do catálogo na execução seguinte.

Escopo

O catálogo cobre os modelos dbt do projeto, isto é, as tabelas e views produzidas pela camada de transformação. Não cobre as tabelas de pouso bruto criadas diretamente pelas rotinas de ingestão, que estão documentadas no catálogo de fontes e nas fichas de pipeline.





O catálogo é um modelo dbt como qualquer outro, `models/metadata/models_metadata.sql`, com uma diferença: em vez de ler tabelas do banco, ele lê o grafo do próprio projeto dbt em tempo de compilação.

A varredura do grafo

Durante a compilação, o dbt expõe a variável `graph`, que contém todos os nós do projeto. O modelo percorre esses nós, filtra os que são do tipo `model` e monta uma linha por modelo encontrado:

```
{% for node in graph.nodes.values() %}
  {% if node.resource_type == 'model' %}
    {% do models_data.append({
      'schema_name':    node.schema,
      'table_name':     node.name,
      'database_name':  node.database,
      'materialization': node.config.materialized,
      'description':    node.description
    }) %}
  {% endif %}
{% endfor %}
```

Trecho de `models/metadata/models_metadata.sql`. A lista de modelos não é digitada: é derivada do grafo a cada execução. Modelos desabilitados não entram no grafo e, portanto, não entram no catálogo.

Carga incremental com chave composta

O modelo é materializado como `incremental`, com `unique_key` composta por `schema_name` e `table_name`, e `on_schema_change='sync_all_columns'`. Cada execução atualiza a linha do modelo em vez de acrescentar uma nova, de forma que a tabela mantém sempre o retrato mais recente de cada modelo.



Quando o catálogo é atualizado

A atualização acontece junto com a transformação: a rotina `minc_cosmos_dag` executa o projeto dbt inteiro todo dia à 01h00, e `models_metadata` é um dos modelos processados nessa passagem. Não existe um processo separado de coleta de metadados que possa ficar para trás.

Propriedade	Valor
Modelo	<code>metadata.models_metadata</code>
Materialização	<code>incremental</code>
Chave única	<code>schema_name + table_name</code>
Mudança de schema	<code>sync_all_columns</code>
Orquestração	<code>minc_cosmos_dag</code> (Airflow + Cosmos)
Frequência	Diária, 01h00 (<code>0 1 * * *</code>)
Fuso do carimbo	<code>America/Sao_Paulo</code> (UTC-3)
Retentativas	2 (<code>default_args</code> da rotina)

Integração com catálogo externo

Os modelos carregam *tags* nativas do dbt, e não `meta.tags`. A escolha é deliberada: é a tag nativa que o conector dbt do OpenMetadata mapeia para tags do catálogo externo. O próprio `models_metadata` é marcado com `infraestrutura`, `metadata` e `governance`, sinalizando que existe para operar o pipeline e não como dado de negócio consumível.



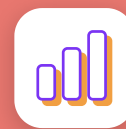


A tabela tem sete campos. Três identificam o modelo, dois descrevem como ele foi construído e dois registram quando e por qual execução foi produzido.

Campo	Descrição	Teste
<code>schema_name</code>	Schema onde o modelo está materializado.	<code>not_null</code>
<code>table_name</code>	Nome da tabela ou view produzida pelo modelo.	<code>not_null</code>
<code>database_name</code>	Banco de dados de destino.	—
<code>materialization</code>	Tipo de materialização: <code>table</code> , <code>view</code> ou <code>incremental</code> .	—
<code>description</code>	Descrição do modelo, extraída do <code>schema.yml</code> correspondente.	—
<code>dt_transform</code>	Data e hora da última transformação, correspondente ao <code>run_started_at</code> da execução.	<code>not_null</code>
<code>run_id</code>	Identificador único da execução do dbt (<code>invocation_id</code>), para rastrear qual rodada gerou a linha.	—

O CAMPO QUE RESPONDE “ESSE DADO ESTÁ FRESCO?”

`dt_transform` é o campo de atualidade do catálogo. Registra o início da execução do dbt que produziu a linha, no fuso `America/Sao_Paulo`, e é a base da dimensão de atualidade descrita no documento de critérios de qualidade dos dados.



Os números abaixo foram apurados em consulta direta à tabela `metadata.models_metadata`, em 24 de agosto de 2026, e conferidos contra os arquivos de configuração do projeto dbt.

Indicador	Valor
Modelos no catálogo	33
Schemas de destino	4
Materializações em uso	3
Última atualização registrada	19/08/2026, 01h10

Os schemas de destino registrados são `agentes`, `cotas`, `cultura` e `metadata`. O schema `cultura` é o schema padrão do ambiente, onde a camada semântica materializa sua tabela de apoio de calendário. O carimbo em `dt_transform` é o que diz a quem consulta se o número que vai citar está fresco.

O QUE O CATÁLOGO REGISTRA — E O QUE ELE NÃO REGISTRA

O catálogo deriva do grafo do projeto: registra o que cada modelo define, com o carimbo da execução que o processou. A existência física de cada tabela é outra pergunta, feita ao schema. Na consulta de 24/08/2026, os domínios de agentes, de metadados e da camada semântica estavam materializados no banco; as tabelas do domínio de cotas aguardavam as planilhas de origem pousarem no ambiente compartilhado — a cadeia de anexos que as carrega é manual e ainda não havia sido executada nesse ambiente.



Distribuição por domínio e camada

Domínio	Camada	Modelos	Materialização
agentes_dbt	bronze	5	incremental
agentes_dbt	silver	1	table
agentes_dbt	gold	4	table
agentes_dbt	views	1	view
cotas_dbt	bronze	8	table
cotas_dbt	silver	8	view
cotas_dbt	gold	4	table
metadata	metadata	1	incremental
camada semântica	cultura	1	table

A materialização é definida por camada em `dbt_project.yml`, e não modelo a modelo. O bronze de `cotas_dbt` é `table` e não `incremental` porque filtra o resíduo de parsing das planilhas, reduzindo milhões de linhas a cerca de 20 mil.

Modelos que aguardam a fonte para entrar no catálogo

Quatro modelos do domínio de cotas estão declarados com `config(enabled=false)` à espera da conclusão da extração do BB Ágil e, por isso, não aparecem no grafo nem no catálogo. A situação está registrada na descrição de cada um, com a instrução de reativá-los em conjunto.

Modelo	Camada	Situação
stg_bbagil	cotas · bronze	Aguarda a rotina <code>extracao_bbagil_dag</code> concluir com sucesso.
bbagil_ente_ano	cotas · silver	Reativa junto com <code>stg_bbagil</code> .
fct_pagamentos_bbagil	cotas · gold	Depende dos dois anteriores.
comparativo_recebido_vs_pago	cotas · gold	Depende de <code>fct_pagamentos_bbagil</code> .

CONSEQUÊNCIA PARA QUEM LÊ OS NÚMEROS

Enquanto esses quatro modelos aguardam a fonte, as tabelas de distribuição de cotas medem valor recebido por entes federados, e não valor pago a pessoas. A ressalva está registrada no documento de exemplos de uso e no código de cada modelo.





O catálogo cobre hoje a dimensão técnica dos metadados. Duas dimensões previstas no produto da Meta 02 ainda não estão registradas, e ambas dependem de decisão, não de código.

Atributo previsto	Situação	O que falta
Origem	Parcial	A linhagem existe no grafo do dbt, mas não é gravada como coluna do catálogo.
Periodicidade	Parcial	Está no agendamento de cada rotina, não no catálogo dos modelos.
Responsável	Ausente	Exige atribuir responsável por domínio de dados — decisão de governança.
Classificação	Ausente	Exige classificar quais tabelas contêm dado pessoal e de que natureza.

COMO ACRESCENTAR SEM QUEBRAR NADA

Os dois atributos ausentes cabem no bloco `meta:` do `schema.yml` de cada modelo, que o dbt já expõe em `node.config.meta`. Isso significa acrescentar duas chaves ao laço de varredura do grafo e duas colunas à tabela, sem mudar a arquitetura do catálogo. A materialização incremental com `on_schema_change='sync_all_columns'` absorve colunas novas sem exigir recarga completa.

Com esses dois campos preenchidos, o catálogo passa a responder às quatro perguntas de governança sobre qualquer tabela da plataforma: de onde veio, quem responde por ela, com que frequência muda e se contém dado pessoal.